

Acceso a bases de datos orientadas a objetos.



Caso práctico

Tras el éxito del proyecto desarrollado para la notaría —en el que María y Juan implementaron una solución basada en bases de datos relacionales utilizando JDBC—, en BK Programación ha llegado un nuevo encargo que supone un reto aún mayor para el equipo.

Una mañana, Ada reúne de nuevo a María y Juan en su despacho. En esta ocasión también se incorpora Ana, ya que el nuevo proyecto requerirá ampliar conocimientos y explorar nuevas tecnologías.

El cliente es una empresa del sector de la madera, especializada en el diseño, fabricación y comercialización de mobiliario. La aplicación que solicitan debe ser capaz de gestionar una gran variedad de piezas, desde elementos simples hasta estructuras complejas que evolucionan a lo largo del proceso de diseño, incorporando transformaciones como pliegues, cortes o deformaciones.

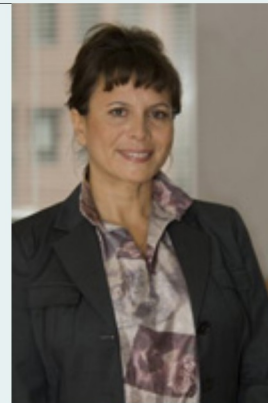
Además, no solo será necesario almacenar estas piezas, sino también gestionar sus relaciones, su evolución en el tiempo y los productos finales que se generan a partir de ellas. Esto implica trabajar con datos muy diversos, con estructuras cambiantes y relaciones complejas entre objetos.

Durante la reunión, Juan y María plantean inicialmente utilizar bases de datos relacionales, como hicieron en el proyecto anterior. Sin embargo, tras analizar los requisitos, Ada señala que este enfoque puede resultar limitado. Las bases de datos relacionales están diseñadas para estructuras tabulares (filas y columnas), lo que dificulta la representación de objetos complejos y sus relaciones dinámicas.

Es entonces cuando surge la necesidad de explorar alternativas más avanzadas. Ada propone investigar el uso de bases de datos orientadas a objetos o bases de datos objeto-relacionales, que permiten almacenar directamente estructuras más cercanas a los objetos utilizados en programación.

Ana, consciente de que participará activamente en este nuevo proyecto, decide repasar los conceptos relacionados con estas tecnologías, ya que serán clave para afrontar con éxito este nuevo desafío.

A partir de este momento, el equipo se adentra en el estudio de nuevas formas de persistencia de datos, ampliando lo aprendido en la unidad anterior y dando un paso más en el desarrollo de aplicaciones profesionales.



 Ministerio de
Educación (Uso educativo
nc)



  Ministerio de Educación y Formación Profesional. (Dominio público)

Materiales formativos de FP Online propiedad del Ministerio de Educación y Formación Profesional.

[Aviso Legal](#) 

1.- Introducción.

Las Bases de Datos Relacionales (BDR) son ideales para aplicaciones tradicionales que soportan tareas administrativas, y que trabajan con datos de estructuras simples y poco cambiantes, incluso cuando la aplicación pueda estar desarrollada en un lenguaje OO y sea necesario un Mapeo Objeto Relacional (ORM)



Q Isabel M. Cruz Granados (Uso

educativo no)

Pero cuando la aplicación requiere otras necesidades, como por ejemplo, soporte multimedia, almacenar objetos muy cambiantes y complejos en estructura y relaciones, este tipo de base de datos no son las más adecuadas. Recuerda, que si queremos representar un objeto y su relaciones en una BDR esto implica que:

- Los objetos deben ser descompuestos en diferentes tablas.
- A mayor complejidad, mayor número de tablas, de manera que se requieren muchos enlaces (joins) para recuperar un objeto, lo cual disminuye dramáticamente el rendimiento.

Las **Bases de Datos Orientadas a Objetos** (BDOO) o Bases de Objetos se integran directamente y sin problemas con las aplicaciones desarrolladas en lenguajes orientados a objetos, ya que **soportan un modelo de objetos puro** y son ideales para almacenar y recuperar datos complejos permitiendo a los usuarios su [navegación](#)  directa (sin un mapeo entre distintas representaciones).

Las Bases de Objetos aparecieron a finales de los años 80 motivadas fundamentalmente por dos razones:

- Las necesidades de los lenguajes de Bases de Datos Orientadas a Objetos (POO), como la necesidad de persistir objetos.
- Las limitaciones de las bases de datos relacionales, como el hecho de que sólo manejan estructuras muy simples (tablas) y tienen poca riqueza semántica



Q Agustin Diaz (CC BY-SA)

Pero como las BDOO no terminaban de asentarse, debido fundamentalmente a la inexistencia de un estándar, y las BDR gozaban y gozan en la actualidad de una gran aceptación, experiencia y difusión, debido fundamentalmente a su gran robustez y al lenguaje SQL, los fabricantes de bases de datos comenzaron a implementar nuevas funcionalidades orientadas a objetos en las BDR existentes. Así surgieron, las bases de datos objeto-relacionales.

Las **Bases de Datos Objeto-Relacionales** (BDOR) son bases de datos relacionales que han evolucionado hacia una base de datos más extensa y compleja, incorporando conceptos del modelo orientado a objetos. Pero en

estas bases de datos **aún existe un mapeo de objetos subyacente**, que es costoso y poco flexible, cuando los objetos y sus interacciones son complejos.

2.- Concepto de bases de datos orientada a objetos.



Caso práctico



Q Ministerio de Educación (Uso educativo [nc](#))

Tras la reunión inicial, Ana decide profundizar en los contenidos necesarios para afrontar el nuevo proyecto. Para ello, recupera sus apuntes del ciclo formativo sobre bases de datos y acceso a datos, y se reúne con Juan para repasarlos.

Mientras revisan la información, comienzan a analizar las características de las bases de datos orientadas a objetos, una tecnología que podría encajar mejor con los

requisitos del proyecto.

—Tiene sentido —comenta Juan—. Estamos trabajando con objetos complejos, con propiedades y comportamientos que cambian con el tiempo. Necesitamos un sistema que nos permita representarlos de forma más natural que con tablas.

Ana asiente mientras toma notas y continúa revisando el material.

—Pero no basta con conocer cómo funcionan —añade Juan—. También tendremos que analizar sus ventajas e inconvenientes. Tenemos que elegir la mejor opción para almacenar los datos de la aplicación.

Ambos saben que la decisión que tomen será clave para el desarrollo del proyecto, por lo que comienzan a estudiar en detalle este tipo de bases de datos, valorando en qué situaciones resultan más adecuadas y qué limitaciones pueden presentar.

De este modo, el equipo da el primer paso para comprender el concepto de bases de datos orientadas a objetos y su aplicación en el desarrollo de software.

Concepto de bases de datos orientadas a objetos

En una base de datos orientada a objetos (BDOO), la información se almacena directamente en forma de **objetos**, del mismo modo que se trabaja en programación orientada a objetos.

Un **objeto** es una entidad identificable que representa un elemento del mundo real y que está compuesto por:

- **Estado:** conjunto de valores de sus atributos (propiedades).
- **Comportamiento:** conjunto de operaciones o métodos que se pueden realizar sobre él.



Q Pixabay (CC0)

Esto permite que los datos no solo se almacenen, sino que también estén asociados a las operaciones que los manipulan, manteniendo una estructura más cercana a la lógica del programa.

Equivalencia con el modelo relacional

Para entender mejor este modelo, se puede establecer la siguiente correspondencia con las bases de datos relacionales:

- Las **entidades** → se corresponden con **clases**
- Las **ocurrencias de entidad (registros)** → se corresponden con **objetos**
- Las **relaciones** → se representan mediante **referencias entre objetos**
- Las **claves primarias** → se sustituyen por **identificadores de objeto (OID)**

Característica principal

La principal característica de las BDOO es que **utilizan un modelo de objetos puro**, lo que implica que:

- Los datos se almacenan como objetos.
- El esquema de la base de datos y el lenguaje de programación comparten las mismas estructuras y tipos.

Esto elimina la necesidad de transformar los datos entre el modelo relacional y el modelo de objetos.

Características de las bases de datos orientadas a objetos

- **Soporte de la programación orientada a objetos**
Permiten aplicar conceptos como encapsulación, herencia, polimorfismo y agregación directamente en la base de datos.
- **Identificador de objeto (OID)**
Cada objeto tiene un identificador único generado por el sistema. Este identificador:
 - Es independiente de los valores del objeto.
 - No cambia aunque cambien sus atributos.
 - Permite distinguir objetos incluso si tienen el mismo contenido.
- **Jerarquía y extensión de tipos**
Se pueden definir nuevos tipos a partir de otros existentes, formando jerarquías de clases, igual que en programación.
- **Objetos complejos**
Los objetos pueden contener otros objetos o estructuras complejas, permitiendo modelar información de forma mucho más realista.
- **Acceso navegacional**
El acceso a los datos se realiza navegando entre objetos mediante referencias, sin necesidad de operaciones complejas como los *joins* de las bases de datos relacionales.
- **Gestión de versiones**
Un mismo objeto puede tener varias versiones, lo que resulta útil en aplicaciones donde los datos evolucionan o se modifican con frecuencia.



Autoevaluación

Señala las opciones correctas. Las bases de datos orientadas a objetos:

- ☐ Permiten almacenar datos junto con su comportamiento.
- ☐ Requieren siempre el uso de claves primarias definidas por el usuario.
- ☐ Utilizan tablas con filas y columnas como estructura principal.
- ☐ Facilitan la representación de objetos complejos.

Mostrar retroalimentación

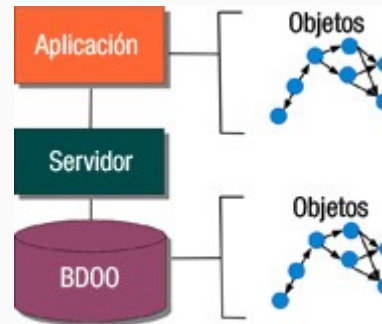
Solución

1. Correcto (#answer-6_58)
2. Incorrecto (#answer-6_85)
3. Incorrecto (#answer-6_87)
4. Correcto (#answer-6_89)

2.1.- Ventajas e inconvenientes.

El uso de una base de datos orientada a objetos (BDOO) puede resultar especialmente adecuado frente a una base de datos relacional cuando la aplicación requiere:

- Gestionar un gran número de tipos de datos diferentes.
- Trabajar con numerosas relaciones entre objetos.
- Manipular objetos con comportamientos complejos.



Q Isabel M. Cruz Granados (Uso educativo
nc)

Una de las principales ventajas de las BDOO es la **transparencia**. Esto significa que el programador puede trabajar directamente con objetos, sin preocuparse de cómo se almacenan o recuperan físicamente los datos. De este modo, se centra en la lógica de la aplicación y no en los detalles de persistencia.

Ventajas

- **Gran capacidad de modelado**
Permiten representar el mundo real de forma más natural gracias al uso de encapsulación, herencia y polimorfismo.
- **Flexibilidad**
La estructura puede evolucionar fácilmente añadiendo nuevas clases o subclases sin afectar significativamente al sistema.
- **Gestión de objetos complejos**
Facilitan el trabajo con estructuras complejas mediante referencias entre objetos, evitando transformaciones adicionales.
- **Alta velocidad de procesamiento**
Los resultados se obtienen directamente como objetos, evitando reconstrucciones costosas como ocurre en bases de datos relacionales.
- **Extensibilidad**
Permiten crear nuevos tipos de datos a partir de los existentes y reutilizar estructuras mediante jerarquías de clases.
- **Reutilización de código**
Aprovechan las ventajas de la programación orientada a objetos, reduciendo el esfuerzo de desarrollo.
- **Control de acceso y concurrencia**
Es posible gestionar el acceso a nivel de objeto, incluso en estructuras jerárquicas completas.
- **Buen rendimiento en sistemas distribuidos**
Funcionan de forma eficiente en entornos cliente/servidor y arquitecturas distribuidas.

Inconvenientes

A pesar de sus ventajas, las BDOO también presentan algunas limitaciones que

deben tenerse en cuenta:

- **Falta de un modelo universal**

No existe un modelo de datos estándar ampliamente aceptado, lo que dificulta su adopción generalizada.

- **Escasez de estándares**

No hay un estándar global consolidado como ocurre con SQL en bases de datos relacionales.

- **Mayor complejidad**

Su estructura puede resultar más difícil de entender y diseñar, especialmente para principiantes.

- **Menor implantación**

Las bases de datos relacionales y objeto-relacionales están mucho más extendidas en el ámbito profesional.

- **Dificultad en la optimización de consultas**

Optimizar el acceso a los datos puede requerir conocer detalles internos de implementación, lo que puede entrar en conflicto con el principio de encapsulación.



Autoevaluación

Señala si la siguiente afirmación es verdadera o falsa.

En una BDOO el resultado de una consulta son objetos, por lo que no es necesario reensamblar los objetos cada vez que se accede a la base de datos.

☒ Verdadero ☐ Falso

Verdadero

Es verdadero, precisamente ésta es una de las ventajas de estos sistemas de almacenamiento.

3.- Modelo de datos orientado a objetos.



Caso práctico

A la mañana siguiente, Juan llega al despacho con varias ideas en mente para comenzar a probar diferentes sistemas de bases de datos orientadas a objetos.

—He estado investigando —comenta Juan— y, a diferencia de las bases de datos relacionales, los gestores orientados a objetos pueden ser bastante distintos entre sí. No existe un estándar tan consolidado como SQL, así que cada sistema tiene sus propias características.

Ana asiente mientras revisa sus apuntes.

—Sí, eso lo vimos en clase —responde—. Probamos varias opciones diferentes para entender cómo funcionaban. Cada una tenía su propia forma de trabajar con los objetos.

Juan continúa:

—Por eso, para nuestro proyecto, he decidido que empecemos con una opción práctica y actual: **ObjectDB**. Es un sistema orientado a objetos que funciona muy bien con Java y nos permitirá probar fácilmente cómo almacenar y recuperar objetos como los que estamos utilizando en la aplicación.

Ana sonríe.

—Perfecto —dice—. Así podremos ver en la práctica cómo se almacenan directamente los objetos sin tener que transformarlos en tablas.

De este modo, el equipo comienza a preparar el entorno de trabajo para experimentar con bases de datos orientadas a objetos y comprobar sus ventajas en aplicaciones reales.



Q Ministerio de Educación (Uso educativo nc)

Para trabajar con bases de datos orientadas a objetos (BDOO), no basta con conocer la programación orientada a objetos, sino que es necesario comprender cómo se representan y almacenan los datos dentro de este tipo de sistemas.

El **modelo de datos orientado a objetos** define cómo se estructuran, relacionan y gestionan los objetos dentro de la base de datos. A diferencia del modelo relacional, que se basa en tablas, este modelo utiliza directamente objetos, manteniendo la misma estructura que en el lenguaje de programación.

Características del modelo de datos en una BDOO



Q Isabel M. Cruz Granados (Uso educativo nc)

■ **Persistencia de objetos**

Los objetos creados en la aplicación pueden almacenarse de forma permanente en la base de datos, manteniendo su estado y sus relaciones.

■ **Identidad de objeto (OID)**

Cada objeto almacenado tiene un identificador único generado por el sistema, independiente de sus atributos, lo que permite distinguirlo de otros objetos.

■ **Encapsulación**

Los objetos almacenan tanto datos como comportamiento, permitiendo acceder a ellos mediante métodos definidos.

■ **Herencia**

Las clases pueden organizarse en jerarquías, permitiendo que los objetos hereden propiedades y métodos de otras clases dentro de la base de datos.

■ **Polimorfismo**

Se pueden aplicar operaciones sobre objetos de diferentes clases de forma uniforme, facilitando la reutilización de código.

Estructura del modelo

En una base de datos orientada a objetos, los elementos principales son:

■ **Clases persistentes**

Definen la estructura de los objetos que se almacenarán en la base de datos.

■ **Objetos persistentes**

Son instancias de las clases que se guardan de forma permanente.

■ **Atributos**

Representan las propiedades de los objetos.

■ **Métodos**

Permiten operar sobre los objetos almacenados.

Relaciones entre objetos

Las relaciones en una BDOO no se basan en claves externas, sino en **referencias directas entre objetos**. Esto permite modelar estructuras complejas de forma natural.

- Un objeto puede contener otros objetos.
- Un objeto puede referenciar a otros objetos relacionados.
- Se pueden crear estructuras jerárquicas o en red.

Diferencia con el modelo relacional

Mientras que en las bases de datos relacionales es necesario transformar los objetos en tablas, en una BDOO los objetos se almacenan directamente, evitando conversiones intermedias.

Esto reduce la llamada **impedancia objeto-relacional**, es decir, la dificultad de adaptar los datos entre el modelo de programación y el modelo de almacenamiento.

Ventaja principal del modelo

La principal ventaja del modelo de datos orientado a objetos es que permite

mantener coherencia entre la aplicación y la base de datos, facilitando el desarrollo de aplicaciones complejas y reduciendo la necesidad de transformaciones de datos.

Ejemplos de SGBOO

- **Db4o de Versant.** Es una BDOO Open Source para Java y .NET. Se distribuye bajo licencia GPL.
- **Matisse.** Es un SGBOO basado en la especificación ODMG, proporciona lenguajes para definición y manipulación de objetos, así como interfaces de programación para C, C++, Eiffel y Java.
- **ObjectDB.** Es una BDOO que ofrece soporte para Java, C++, y Python entre otros lenguajes. No es un producto libre, aunque ofrecen versiones de prueba durante un periodo determinado.
- **EyeDB.** Es un SGBOO basado en la especificación ODMG, proporciona lenguajes para definición y manipulación de objetos, e interfaces de programación para C++ y Java. Se distribuye bajo licencia GNU y es software libre.
- Neodatis, ObjectStore y GemStone. Son otros SGBDOO.

3.1.- Ejemplo práctico de clase persistente en una BDOO

Para comprender mejor el modelo de datos orientado a objetos, vamos a ver un ejemplo sencillo. Supongamos que la empresa necesita almacenar información sobre los muebles que fabrica.

En una base de datos orientada a objetos, los objetos de la aplicación pueden almacenarse directamente, manteniendo sus atributos, sus métodos y sus relaciones con otros objetos.

Podemos partir de una clase general llamada **Producto** y crear una subclase llamada **Mueble**.

Clase Producto

```
public class Producto {
    private String codigo;
    private String nombre;
    private double precio;

    public Producto(String codigo, String nombre, double
        this.codigo = codigo;
        this.nombre = nombre;
        this.precio = precio;
    }

    public String getCodigo() {
        return codigo;
    }

    public String getNombre() {
        return nombre;
    }

    public double getPrecio() {
        return precio;
    }

    public void setPrecio(double precio) {
        this.precio = precio;
    }

    public void mostrarInformacion() {
        System.out.println("Código: " + codigo);
        System.out.println("Nombre: " + nombre);
        System.out.println("Precio: " + precio);
    }
}
```

Clase Mueble

```
public class Mueble extends Producto {
```



Q Isabel M. Cruz Granados (Uso educativo no)

```

    private String material;
    private String color;

    public Mueble(String codigo, String nombre, double p
        super(codigo, nombre, precio);
        this.material = material;
        this.color = color;
    }

    public String getMaterial() {
        return material;
    }

    public String getColor() {
        return color;
    }

    @Override
    public void mostrarInformacion() {
        super.mostrarInformacion();
        System.out.println("Material: " + material);
        System.out.println("Color: " + color);
    }
}

```

Uso de los objetos en la aplicación

```

public class Main {
    public static void main(String[] args) {
        Mueble m1 = new Mueble("M001", "Silla nórdica",
        Mueble m2 = new Mueble("M002", "Mesa auxiliar",

        m1.mostrarInformacion();
        System.out.println("-----");
        m2.mostrarInformacion();
    }
}

```

Conclusión: en una BD00, los objetos de la aplicación se almacenan directamente en la base de datos, sin necesidad de convertirlos previamente en tablas. Esto facilita el trabajo con herencia, referencias entre objetos y estructuras complejas.

¿Qué ocurre en una BD00?

En una base de datos relacional, para guardar estos datos habría que transformar los objetos en registros de una o varias tablas. Sin embargo, en una **base de datos orientada a objetos**, los objetos **m1** y **m2** se pueden almacenar directamente como objetos persistentes.

Esto significa que:

- Se conserva la estructura de clases.
- Se mantiene la herencia entre **Producto** y **Mueble**.
- Los objetos pueden recuperarse posteriormente con sus atributos y su comportamiento.

- No es necesario transformar la información en filas y columnas.

Ventaja del modelo

Este ejemplo muestra cómo una BD OO permite trabajar con una estructura de datos mucho más cercana al diseño de la aplicación. De este modo, el programador puede almacenar y recuperar objetos complejos de forma más natural.



Autoevaluación

Señala si la siguiente afirmación es verdadera o falsa.

En una base de datos orientada a objetos, los objetos se almacenan directamente sin necesidad de transformarlos en tablas.

☒ Verdadero ☐ Falso

Verdadero

Es verdadero, efectivamente en una base de datos orientada a objetos, los objetos se almacenan directamente sin necesidad de transformarlos en tablas.

3.2.- Instalación del gestor de objetos ObjectDB.

1. Descargar e instalar

ObjectDB se distribuye como un archivo ZIP. No requiere instalación: basta con descargarlo y descomprimirlo en una carpeta del equipo.

<https://www.objectdb.com/download>

No es necesario ejecutar ningún programa de instalación.



Q Luis Perez (CC BY-NC-SA)

2. Crear un proyecto Java

Puedes crear el proyecto en cualquier entorno de desarrollo como Eclipse.

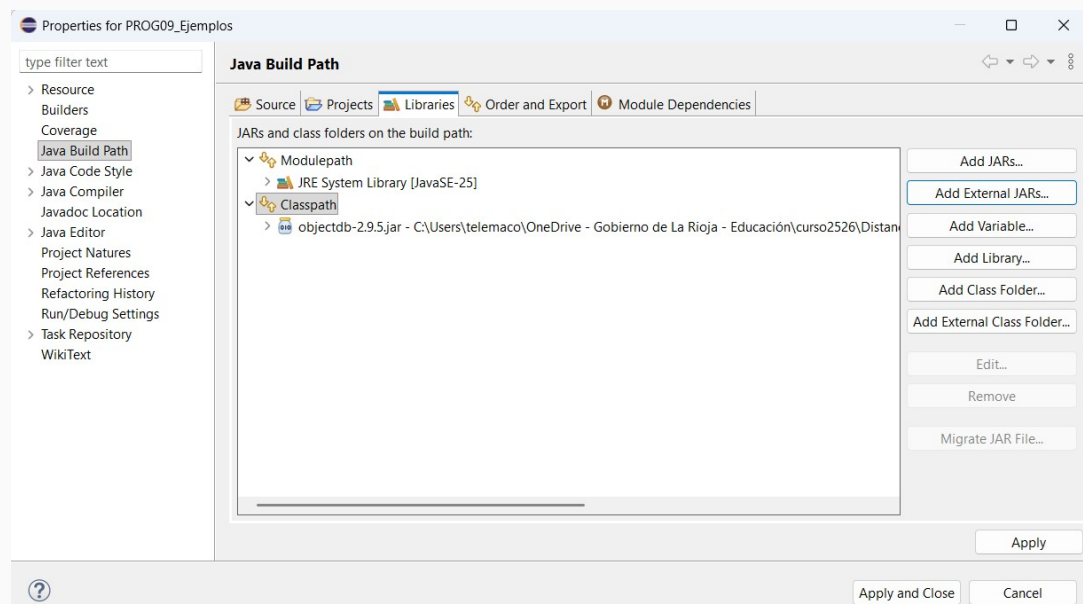
ObjectDB funciona especialmente bien con Java mediante JPA, Java Persistence API.

3. Añadir la librería al proyecto

Añade el archivo JAR de ObjectDB al *classpath* del proyecto. Para ello, accede a la carpeta que has descomprimido en el paso anterior y localiza el archivo **.jar** en la siguiente ruta:

objectdb-2.9.5/bin/objectdb-2.9.5.jar

Este archivo debe incorporarse al proyecto para poder utilizar las funcionalidades de ObjectDB desde Java.



4. Crear la clase persistente

En ObjectDB es habitual utilizar JPA. Para ello, se debe anotar la clase con **@Entity** y definir un identificador con **@Id**.

Ejemplo de clase Mueble:

```
import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.Id;
```

```

@Entity
public class Mueble {
    @Id @GeneratedValue
    private long id;
    private String nombre;
    private double precio;
    private String material;
    private String color;

    public Mueble() {
    }

    public Mueble( String nombre, double precio,

        this.nombre = nombre;
        this.precio = precio;
        this.material = material;
        this.color = color;
    }

    public long getId() { return id; }
    public String getNombre() { return nombre; }
    public double getPrecio() { return precio; }
    public String getMaterial() { return materia
    public String getColor() { return color; }

    public void setPrecio(double precio) { this.

    @Override
    public String toString() {
        return id + " - " + nombre + " (" + prec
    }
}

```

5. Guardar objetos en la base de datos

Ejemplo de persistencia de un objeto:

```

import javax.persistence.EntityManager;
import javax.persistence.EntityManagerFactory;
import javax.persistence.Persistence;

public class Main {
    public static void main(String[] args) {
        EntityManagerFactory emf =
            Persistence.createEntityManagerFactory("$obj
        EntityManager em = emf.createEntityManager();

        em.getTransaction().begin();

        Mueble m1 = new Mueble( "Silla nórdica", 129.99,
        em.persist(m1);
        Mueble m2 = new Mueble("Mesa auxiliar", 89.50, "
        em.persist(m2);
    }
}

```

```

        em.getTransaction().commit();

        em.close();
        emf.close();
    }
}

```

6. Consultar los objetos guardados

Ejemplo de consulta:

```

import javax.persistence.EntityManager;
import javax.persistence.EntityManagerFactory;
import javax.persistence.Persistence;

import java.util.List;

public class ConsultaMuebles {
    public static void main(String[] args) {
        EntityManagerFactory emf =
            Persistence.createEntityManagerFactory("$obj
            EntityManager em = emf.createEntityManager();

        List muebles =
            em.createQuery("SELECT m FROM Mueble m", Mue

        for (Mueble m : muebles) {
            System.out.println(m);
        }

        em.close();
        emf.close();
    }
}

```

En las siguientes secciones se explica paso a paso lo realizado en este ejemplo.

4.- Conexión a una base de datos orientada a objetos.



Caso práctico



🔍 Ministerio de Educación (Uso educativo nc)

un ardua sesión de trabajo, familiarizarse con el API de la base de objetos Db4o.

Ana no puede negar su entusiasmo con el hecho de participar en el desarrollo del nuevo proyecto. Cuando estudió Acceso a Datos en el Ciclo Formativo, nunca pensó que llegaría a trabajar tan de cerca con las bases de datos avanzadas.

Ha pasado en el descanso a ver a su amigo **Antonio** y ponerlo al día de los avances que va realizando junto a **Juan**. Hoy le espera

Una vez instalado el sistema gestor de bases de datos orientado a objetos, el siguiente paso es establecer la conexión entre la aplicación y la base de datos.

A diferencia de las bases de datos relacionales, donde se utiliza JDBC, en muchas bases de datos orientadas a objetos (como ObjectDB) se emplea **JPA (Java Persistence API)** para gestionar la conexión y la persistencia de los objetos.

Elementos necesarios para la conexión

Para conectar una aplicación Java con una BDOO se necesitan los siguientes elementos:

- **Unidad de persistencia**

Definida en el archivo *persistence.xml*, indica cómo se conecta la aplicación con la base de datos.

- **EntityManagerFactory**

Objeto encargado de crear las conexiones con la base de datos.

- **EntityManager**

Permite realizar operaciones sobre los objetos (guardar, consultar, eliminar, etc.).

- **Transacciones**

Garantizan que las operaciones sobre la base de datos se realicen correctamente.

Proceso de conexión

El proceso de conexión a una base de datos orientada a objetos sigue los siguientes pasos:

1. Crear la unidad de persistencia (*persistence.xml*).

2. Obtener un **EntityManagerFactory**.
3. Crear un **EntityManager**.
4. Iniciar una transacción.
5. Realizar operaciones sobre los objetos.
6. Confirmar la transacción.
7. Cerrar la conexión.

Ejemplo de conexión

Ejemplo básico en Java:

```
import javax.persistence.EntityManager;
import javax.persistence.EntityManagerFactory;
import javax.persistence.Persistence;

public class Main {
    public static void main(String[] args) {
        EntityManagerFactory emf =
            Persistence.createEntityManagerFactory("$obj
            EntityManager em = emf.createEntityManager();

        em.getTransaction().begin();

        Mueble m1 = new Mueble( "Silla nórdica", 129.99,
        em.persist(m1);
        Mueble m2 = new Mueble("Mesa auxiliar", 89.50, "
        em.persist(m2);

        em.getTransaction().commit();

        em.close();
        emf.close();
    }
}
```

Diferencia con JDBC

En las bases de datos relacionales, la conexión se realiza mediante JDBC, donde el programador debe trabajar directamente con consultas SQL.

Sin embargo, en una base de datos orientada a objetos:

- No es necesario escribir consultas SQL para operaciones básicas.
- Se trabaja directamente con objetos.
- La conexión está integrada dentro del modelo de persistencia.

Ventaja principal

El uso de JPA en bases de datos orientadas a objetos permite una **gestión más sencilla y natural de la persistencia**, ya que el programador trabaja directamente con objetos sin preocuparse por los detalles de la conexión o del almacenamiento físico.



Autoevaluación

Señala si la siguiente afirmación es verdadera o falsa.

Un `ObjectContainer` es un interfaz que proporciona, entre otros, los métodos `store()`, `queryByExample()`, `close()` y `delete()`.

☒ Verdadero ☐ Falso

Verdadero

Es verdadero, es el interfaz que permite realizar las principales tareas con la base de datos.

5.- Almacenamiento de objetos.



Caso práctico



Q Ministerio de Educación (Uso educativo nc)

Almacenamiento de objetos

Una vez establecida la conexión con la base de datos orientada a objetos, el siguiente paso es almacenar los objetos creados en la aplicación.

Ana y Juan comienzan a probar cómo guardar los muebles que han creado en su programa. A diferencia de las bases de datos relacionales, no necesitan transformar los objetos en tablas, sino que pueden almacenarlos directamente.

—Esto es mucho más natural —comenta Ana—. Estamos guardando directamente los objetos que hemos creado en el programa.

—Exacto —responde Juan—. Eso es precisamente lo que hace que las bases de datos orientadas a objetos sean tan útiles en aplicaciones complejas.

El proceso de almacenamiento en una BDOO se conoce como **persistencia de objetos**. Consiste en guardar objetos en la base de datos para poder recuperarlos posteriormente manteniendo su estado y sus relaciones.

Proceso de almacenamiento

Para almacenar un objeto en una base de datos orientada a objetos se siguen los siguientes pasos:

1. Crear el objeto en la aplicación.
2. Iniciar una transacción.
3. Persistir el objeto mediante el **EntityManager**.
4. Confirmar la transacción.

Ejemplo de almacenamiento de un objeto

Ejemplo en Java:

```
import jakarta.persistence.EntityManager;
import jakarta.persistence.EntityManagerFactory;
import jakarta.persistence.Persistence;

public class GuardarMueble {
    public static void main(String[] args) {

        EntityManagerFactory emf =
            Persistence.createEntityManagerFactory("$obj
        EntityManager em = emf.createEntityManager();
```



Q Abraham Pérez Pérez (CC BY-NC-SA)

```

        // Crear objeto
        Mueble m1 = new Mueble("M003", "Mesa comedor", 2

        // Iniciar transacción
        em.getTransaction().begin();

        // Guardar objeto
        em.persist(m1);

        // Confirmar transacción
        em.getTransaction().commit();

        em.close();
        emf.close();
    }
}

```

Almacenamiento de varios objetos

Es posible almacenar múltiples objetos dentro de la misma transacción:

```

em.getTransaction().begin();

Mueble m1 = new Mueble("M004", "Estantería", 150.00, "Ma
Mueble m2 = new Mueble("M005", "Escritorio", 220.00, "Ma

em.persist(m1);
em.persist(m2);

em.getTransaction().commit();

```

Aspectos importantes

- Los objetos deben estar definidos como **entidades (@Entity)**.
- Cada objeto debe tener un identificador único (**@Id**).
- Las operaciones de almacenamiento deben realizarse dentro de una **transacción**.
- Los objetos se almacenan con toda su estructura (atributos y relaciones).

Diferencia con bases de datos relacionales

En una base de datos relacional, almacenar un objeto implica convertirlo en filas de una o varias tablas.

En una BD00, el objeto se guarda directamente, lo que simplifica el proceso y evita transformaciones intermedias.

Ventaja principal

El almacenamiento en bases de datos orientadas a objetos permite trabajar de forma más natural con los datos, ya que se mantienen las estructuras de programación sin necesidad de adaptación.



Autoevaluación

Señala si la siguiente afirmación es verdadera o falsa.

El lenguaje OQL incluye un rico repertorio de sentencias para modificar el estado de los objetos.

☐ Verdadero ☐ Falso

Falso

Es falso, ya que lo normal es utilizar para ello los propios métodos del objeto.

6.- Consultas sobre una base de datos orientada a objetos.



Caso práctico



Q Ministerio de Educación (Uso educativo nc)

Una vez que Ana y Juan han conseguido almacenar los objetos en la base de datos, el siguiente paso es poder recuperarlos y trabajar con ellos.

—Ahora viene lo importante —dice Ana—. ¿Cómo consultamos los muebles que hemos guardado?

Juan responde:

—En las bases de datos orientadas a objetos no trabajamos directamente con tablas, sino con objetos. Para ello utilizamos un lenguaje de consultas adaptado al modelo de objetos.

Lenguaje de consultas: JPQL

En sistemas como ObjectDB se utiliza **JPQL (Java Persistence Query Language)**, un lenguaje de consultas similar a SQL, pero orientado a objetos.

En lugar de trabajar con tablas y columnas, JPQL trabaja con **clases y atributos**.

Ejemplo de consulta

Consulta de todos los muebles:

```
List<Mueble> muebles =  
    em.createQuery("SELECT m FROM Mueble m", Mueble.class)  
Esta consulta devuelve todos los objetos de tipo Mueble almacenados en la base de datos.
```

Consulta con condición

Ejemplo: obtener muebles de un determinado material

```
List<Mueble> muebles =  
    em.createQuery("SELECT m FROM Mueble m WHERE m.mater  
        .setParameter("material", "Madera")  
        .getResultList();
```

En este caso, se filtran los objetos en función de uno de sus atributos.

Recorrido de resultados

```
for (Mueble m : muebles) {  
    System.out.println(m.getNombre() + " - " + m.getPrec  
}
```

Diferencias entre JPQL y SQL

Aunque JPQL es similar a SQL, existen diferencias importantes:

SQL (Relacional)	JPQL (Orientado a objetos)
------------------	----------------------------

Trabaja con tablas	Trabaja con clases
Trabaja con filas	Trabaja con objetos
SELECT * FROM muebles	SELECT m FROM Mueble m
Se utilizan JOIN para relaciones	Se navega entre objetos mediante referencias

Acceso navegacional

Una de las grandes ventajas de las BDOO es el **acceso navegacional**. Esto significa que podemos acceder a los objetos relacionados directamente, sin necesidad de realizar uniones complejas.

Por ejemplo, si un objeto **Mueble** tiene un proveedor asociado:

```
System.out.println(mueble.getProveedor().getNombre());
```

No es necesario realizar un JOIN, ya que la relación se mantiene directamente entre objetos.

Ventaja principal

Las consultas en bases de datos orientadas a objetos permiten trabajar de forma más cercana al modelo de programación, lo que simplifica el desarrollo y mejora la comprensión del código.



Autoevaluación

Señala si la siguiente afirmación es verdadera o falsa.

En una base de datos objeto-relacional, las columnas de una tabla pueden almacenar tipos estructurados.

☐ Verdadero ☐ Falso

Verdadero

Es verdadero, esa es una de las nuevas características de este tipo de bases de datos.